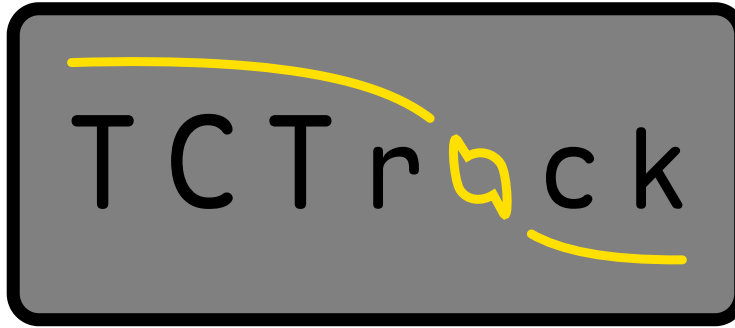




TCTrack: Working with researchers to enable FAIR tropical cyclone tracking

Sam Avis Jack Atkinson Simon Sadler Surbhi Goel Charles Powell
Alison Ming

Institute of Computing for Climate Science, University of Cambridge, UK

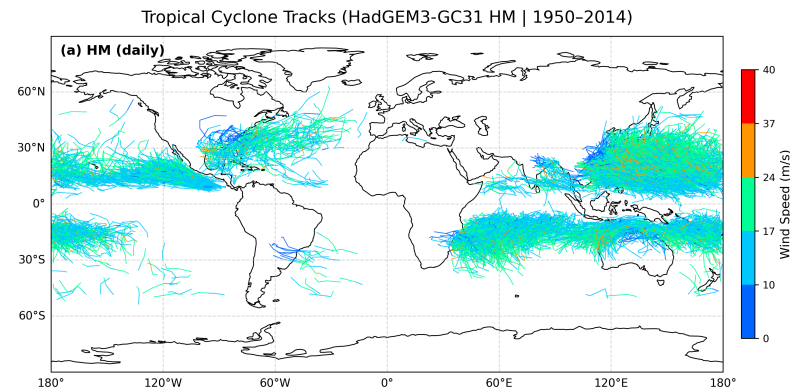
2026-09-10

The starting problem

Topic: Identifying tropical cyclone tracks from gridded data (simulations, forecasts, historical reanalysis data)

Problem:

- Multiple tracking algorithms exist, but give different results.
- We want a code that implements them with a unified interface.



Tropical cyclone tracks identified in climate-model data

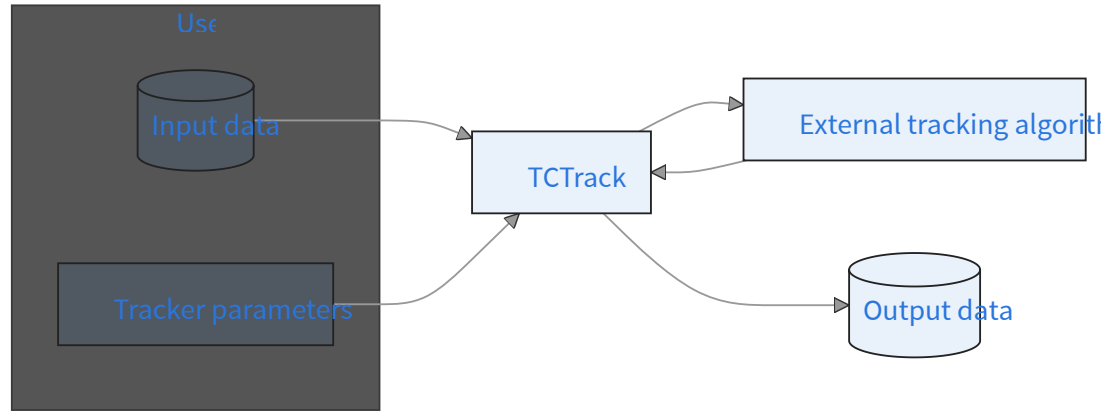
Applying RSE expertise - Don't reinvent the wheel

Initial plan: Reimplement tracking algorithms in Python - researchers did not have full knowledge of existing domain software.

Also, existing codes were not necessarily well documented / work on all systems.

New plan: Understand and build on what is already there to improve code sustainability - RSE expertise needed.

Applying RSE expertise - The framework



- TCTrack only needs an interface and parameters for each tracking algorithm.
- Easily extensible to encourage community contributions.

Building domain experience

To wrap the tracking algorithms, we had to gain an understanding of them rather than treating them as black boxes.

We were able to identify themes across algorithms.

e.g. Broadly 3 main steps:



Building domain experience

- More understanding of code
 - > can expose more parameters and functionality.
- Equally, we want to keep it simple to use - e.g. default parameter sets.

```
detect_params = te.TEDetectParameters(  
    search_by_min="msl",  
    time_filter="6hr",  
    merge_dist=6.0,  
    closed_contours=[  
        te.TEContour(var="msl", delta=200.0, dist=5.5),  
        te.TEContour(var="_DIFF(zg(300hPa),zg(500hPa))", delta  
    ],  
    output_commands=[  
        te.TEOutputCommand(var="msl", operator="min"),  
        te.TEOutputCommand(var="orog", operator="max"),
```

```
detect_params, stitch_params = te.parameter_set("UZ")
```

But to do this we need a good understanding of how the algorithms are used in research.

FAIR outputs: the problem

```
start      2 1950      1      4      0
  54.67 -22.15  16.65  999.23  0.00027446
  55.72 -24.49  17.38  997.83  0.00031078
start     13 1950      1      6      0
  67.32  -9.26  17.90 1002.11  0.00065708
  67.68  -9.26  15.32 1003.88  0.00044106
...
```

```
track_id, year, month, day, hour, i, j, lon, la
0, 1950, 1, 1, 6, 572, 320, 201.269531, -14.882
0, 1950, 1, 1, 12, 569, 318, 200.214844, -15.35
0, 1950, 1, 1, 18, 567, 317, 199.511719, -15.58
0, 1950, 1, 2, 0, 565, 320, 198.808594, -14.882
...
```

```
netcdf ff_trs.test_out {
dimensions:
    tracks = 703 ;
    record = UNLIMITED ; // (78 currently)
variables:
    int TRACK_ID(tracks) ;
        TRACK_ID:add_fld_num = 0 ;
        TRACK_ID:tot_add_fld_num = 0 ;
        TRACK_ID:loc_flags = "" ;
    int FIRST_PT(tracks) ;
```

- Different format for every algorithm
- Unclear what each value represents
- Limited metadata
- Cannot reproduce

FAIR outputs: standardised format

All TCTrack outputs are converted into CF-Compliant NetCDF.

Again build on existing software by heavily relying on cf-python
(Thanks to the NCAS RSE team!)

```
netcdf my_tracks {  
  dimensions:  
    trajectory = 36 ;  
    observation = 60 ;  
  variables:  
    int64 trajectory(trajectory) ;  
        trajectory:standard_name = "trajectory" ;  
        trajectory:cf_role = "trajectory_id" ;  
        trajectory:long_name = "trajectory index" ;  
    int64 observation(observation) ;  
        observation:standard_name = "observation" ;  
        observation:long_name = "observation index" ;  
    double time(trajectory, observation) ;  
        time:standard_name = "time" ;  
        time:long_name = "time" ;  
        time:units = "days since 1970-01-01" ;  
        time:calendar = "360_day" ;  
    double lat(trajectory, observation) ;  
        lat:standard_name = "lat" ;  
        lat:long_name = "latitude" ;  
}
```

FAIR outputs: metadata

Included metadata:

- Variable metadata (copied from input files)
- TCTrack metadata
 - Version
 - Tracking algorithm
 - Tracking parameters

Solves the familiar problem: “How did we get this result?”

FAIR outputs: reproducibility

The metadata is machine-readable, further simplifying reproduction:

```
from tctrack.utils import load_tracker_metadata
tracker, params = load_tracker_metadata("tracks_output.nc")
tracker.run_tracker("tracks_output_2.nc")
```

Conclusions

1. **RSE support** helped to create a more useful software for the community.
2. **Domain knowledge** helped us improve ease of use for researchers' workflows.
3. **FAIR outputs** improve the usability of research outputs.

[github.com/Cambridge-
ICCS/TCTrack](https://github.com/Cambridge-ICCS/TCTrack)
tctrack.readthedocs.io

